House Price Predictor

Building and Evaluating a Linear Regression Model

California Housing dataset | Python, scikit-learn

Project overview

Goal: predict median house value (in $100,000s) for California districts from eight numeric features, using ordinary least squares linear regression, and evaluate it with MAE, RMSE and R^2.

Step-by-step plan

1. Create a virtualenv and install pandas, numpy, scikit-learn, matplotlib, seaborn, jupyter.
2. Load the dataset (sklearn.datasets.fetch_california_housing) or a Kaggle CSV.
3. Perform EDA: distributions, correlations, missing values.
4. Select features and split the data (train_test_split).
5. Train LinearRegression; evaluate using MAE, RMSE, R^2.
6. Plot predicted vs actual scatter and residuals.
7. Save the notebook (.ipynb) and a short PDF slide deck summarizing findings.

Metrics used

Metric	Meaning
MAE	Average absolute error, in $100k units. Easy to interpret.
RMSE	Square root of mean squared error. Penalizes large misses more.
R-squared	Share of price variance explained by the model (1.0 = perfect).

Notes on this document

This PDF contains the full project code, organized by step. The dataset must be downloaded by scikit-learn at run time, so numerical results and plots are produced when you run the notebook (`house_price_linear_regression.ipynb`) on a machine with internet access. Its final cell also generates a results slide deck, `findings_deck.pdf`, from your real metrics.

Known characteristics of this dataset: the target is capped at 5.0 ($500k), and some features have heavy outliers, both of which limit how well a linear model fits.

Full code

Cell 1

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

sns.set_theme(style="whitegrid")
os.makedirs("figs", exist_ok=True)
```

1. Load data

Cell 2

```
housing = fetch_california_housing(as_frame=True)
df = housing.frame.copy()
print(df.shape)
df.head()
```

2. Exploratory data analysis

Cell 3

```
print("Missing values per column:")
print(df.isna().sum())
df.describe().T
```

Cell 4

```
df.hist(bins=40, figsize=(12, 8), color="#2E6F95")
plt.suptitle("Feature and target distributions", y=1.02)
plt.tight_layout()
plt.savefig("figs/distributions.png", dpi=150, bbox_inches="tight")
plt.show()
```

Cell 5

```
corr = df.corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Correlation matrix")
plt.tight_layout()
plt.savefig("figs/correlations.png", dpi=150, bbox_inches="tight")
plt.show()

target_corr = corr["MedHouseVal"].drop("MedHouseVal").sort_values(ascending=False)
target_corr
```

What to look for: `MedInc` (median income) should correlate most strongly with price. `AveRooms`/`AveBedrms` and `AveOccup`/`Population` contain extreme outliers, and `MedHouseVal` is capped at 5.0, which limits how well any linear model can fit the top end.

3. Features and train/test split

Cell 6

```
X = df.drop(columns="MedHouseVal")
y = df["MedHouseVal"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(X_train.shape, X_test.shape)
```

4. Train and evaluate

Cell 7

```
model = LinearRegression()
model.fit(X_train, y_train)

def evaluate(y_true, y_pred):
    return {
        "MAE": mean_absolute_error(y_true, y_pred),
        "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
        "R2": r2_score(y_true, y_pred),
    }

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
metrics = pd.DataFrame({"train": evaluate(y_train, train_pred),
                        "test": evaluate(y_test, test_pred)}).T
print(metrics.round(4))

coefs = pd.Series(model.coef_, index=X.columns).sort_values()
print("\nIntercept:", round(model.intercept_, 4))
coefs.round(4)
```

Cell 8

```
fig, ax = plt.subplots(figsize=(7, 4))
coefs.plot(kind="barh", color="#2E6F95", ax=ax)
ax.set_title("Linear regression coefficients (unscaled)")
plt.tight_layout()
plt.savefig("figs/coefficients.png", dpi=150, bbox_inches="tight")
plt.show()
```

5. Predicted vs actual and residuals

Cell 9

```
residuals = y_test - test_pred

fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

axes[0].scatter(y_test, test_pred, s=6, alpha=0.3, color="#2E6F95")
lims = [min(y_test.min(), test_pred.min()), max(y_test.max(), test_pred.max())]
axes[0].plot(lims, lims, "r--", label="Perfect prediction")
axes[0].set(xlabel="Actual ($100k)", ylabel="Predicted ($100k)", title="Predicted vs actual")
axes[0].legend()

axes[1].scatter(test_pred, residuals, s=6, alpha=0.3, color="#2E6F95")
axes[1].axhline(0, color="r", linestyle="--")
axes[1].set(xlabel="Predicted ($100k)", ylabel="Residual", title="Residuals vs predicted")

sns.histplot(residuals, bins=50, kde=True, ax=axes[2], color="#2E6F95")
axes[2].set(xlabel="Residual ($100k)", title="Residual distribution")

plt.tight_layout()
plt.savefig("figs/diagnostics.png", dpi=150, bbox_inches="tight")
plt.show()
```

Reading the plots: points hugging the red line mean good predictions. The horizontal band at actual = 5.0 is the price cap. A funnel shape in the residual plot means errors grow with price (heteroscedasticity), a sign that a linear model is too simple here. Natural next steps: scale features, add log-transformed or interaction features, or try tree-based models.

6. Save findings as a short PDF deck

Cell 10

```
from reportlab.lib.pagesizes import landscape, letter
```

```
from reportlab.pdfgen import canvas
from reportlab.lib.utils import ImageReader
from reportlab.lib import colors

W, H = landscape(letter)
NAVY, BLUE, GREY = colors.HexColor("#1B2A41"), colors.HexColor("#2E6F95"), colors.HexColor("#555555")
c = canvas.Canvas("findings_deck.pdf", pagesize=(W, H))

def title(text, sub=None):
    c.setFillColor(NAVY); c.setFont("Helvetica-Bold", 28); c.drawString(50, H - 70, text)
    c.setStrokeColor(BLUE); c.setLineWidth(3); c.line(50, H - 82, 170, H - 82)
    if sub:
        c.setFillColor(GREY); c.setFont("Helvetica", 14); c.drawString(50, H - 108, sub)

def bullets(items, x=50, y=None, size=16, gap=26):
    y = y or H - 150
    c.setFont("Helvetica", size)
    for t in items:
        c.setFillColor(BLUE); c.circle(x + 4, y + 5, 3, fill=1, stroke=0)
        c.setFillColor(colors.black); c.drawString(x + 18, y, t); y -= gap

def image(path, x, y, w):
    img = ImageReader(path); iw, ih = img.getSize()
    c.drawImage(img, x, y, width=w, height=w * ih / iw)

def footer(n):
    c.setFillColor(GREY); c.setFont("Helvetica", 10); c.drawRightString(W - 40, 25, f"{n}")

t = metrics.loc["test"]
top_pos, top_neg = coefs.idxmax(), coefs.idxmin()

# 1. Title
c.setFillColor(NAVY); c.rect(0, 0, W, H, fill=1, stroke=0)
c.setFillColor(colors.white); c.setFont("Helvetica-Bold", 38)
c.drawString(60, H / 2 + 20, "House Price Predictor")
c.setFont("Helvetica", 20); c.drawString(60, H / 2 - 20, "Linear Regression on California Housing")
c.showPage()

# 2. Data & EDA
title("Data and exploration", f"{df.shape[0]:,} districts, {X.shape[1]} features, target in $100k")
bullets([
    f"Missing values: {int(df.isna().sum().sum())}",
    f"Strongest correlate with price: {target_corr.index[0]} (r = {target_corr.iloc[0]:.2f})",
    "Target is capped at 5.0 ($500k), which truncates the high end",
    "Several features (AveRooms, AveOccup, Population) have heavy outliers",
], y=H - 150)
image("figs/correlations.png", W - 400, 60, 340)
footer(2); c.showPage()

# 3. Model & metrics
title("Model and results", "LinearRegression, 80/20 split, random_state=42")
bullets([
    f"Test MAE:  {t['MAE']:.3f}  (about ${t['MAE'] * 100_000:,.0f})",
    f"Test RMSE: {t['RMSE']:.3f}  (about ${t['RMSE'] * 100_000:,.0f})",
    f"Test R-squared: {t['R2']:.3f}",
    f"Train R-squared: {metrics.loc['train', 'R2']:.3f}  ({'small gap vs test, no sign of overfitting' if metrics.loc['tr
    f"Largest positive coefficient: {top_pos};  largest negative: {top_neg}",
], y=H - 150)
image("figs/coefficients.png", 50, 60, 400)
footer(3); c.showPage()

# 4. Diagnostics
title("Predicted vs actual, and residuals")
image("figs/diagnostics.png", 40, 200, W - 80)
c.setFillColor(colors.black); c.setFont("Helvetica", 14)
c.drawString(50, 150, "Predictions track the diagonal loosely; the flat band at 5.0 is the price cap.")
c.drawString(50, 126, "Residual spread grows with predicted price, so a purely linear fit has limits.")
footer(4); c.showPage()

# 5. Takeaways
title("Takeaways and next steps")
bullets([
```

```
    f"The model explains about {t['R2'] * 100:.0f}% of variance in held-out prices",
    "Income and location (latitude/longitude) drive most of the signal",
    "Baseline is solid and interpretable, but errors are large for expensive areas",
    "Next: scale/log-transform features, add interactions, try Ridge/Lasso",
    "Then compare against Random Forest or Gradient Boosting",
], y=H - 150)
footer(5); c.showPage()

c.save()
print("Saved findings_deck.pdf")
```